1.3.8 Console IO Functions

Show Lecture Video

"IO" stands for "input and output". Here we will consider so-called console IO functions. The **console** is a text area where a program's print output goes. It's also where the user can enter input to a program.

Console Output with print()

We have already seen `print()`. We use it to print output into the console. One small addition: you can provide multiple arguments to `print()`, and it will print each of those values, each separated by a single space, like so:

```
1 name = 'David'
2 print('My name is', name)
```

Run

Run the above code, and see how it prints `My name is David`.

Note that `print()` takes any number of comma-separated values and prints a line of text with each value separated by one space. For example:

```
1 x = 3
2 y = 2
3 print(x, 'plus', y, 'equals', x+y) # Prints: 3 plu
```

Run

Checkpoint 1

What does the following code print?

```
x = 'a'
y = 'b'
z = 'c'
print(x, y, z)
```

- 'a b c'
- a, b, c
- abc
- 'abc'
- 'a, b, c'
- a b c

Submit

Console Input with input()

To get input from the user in the console, we use the `input()` function, like so:

```
1 name = input('Enter your name: ')
2 print('Your name is', name)
```

Run

First, see how `input()` takes one argument, the **prompt**. In this case, `'Enter your name: '`. Run this code, and see how that prompt is shown, and then the program waits for you to enter some text. Once you press enter, `input()` returns the text you entered. In the code above, that is then assigned to the variable `name`, which we can then use like any other variable.

> **Question:** What will the example above print if you press enter without typing anything in the prompt?
>
> Show answer

This time, let's try (and fail) to input an integer:

```
1 dogYears = input("Enter your dog's age in years:
2 humanYears = 7 * dogYears
3 print("Your dog's age in human years is", humanYe
```

Run

Run the code and enter `5` as your dog's age in years. In response, the program says: `Your dog's age in human years is 5555555`. Oh no!

The problem is that `input()` returns a string, not an int. So instead of computing `7 * 5` to get `35`, the code computed `7 * '5'` to get `'5555555'`.

The fix is easy: just use `int()` to convert what `input()` returns into an integer, like so:

```
1 dogYears = int(input("Enter your dog's age in yea
2 humanYears = 7 * dogYears
3 print("Your dog's age in human years is", humanYe
```

Run

Problem solved!

Question: Look carefully at the previous example, and see that we used double-quotes ("). Why did we use double-quotes in the previous example?

Show answer

Checkpoint 2

What will happen if you don't input a number in the example above?

- ○ **It will not crash, and it will print `Your dog's age in human years is.`**
- ○ **It will crash on line 1.**
- ○ **It will crash on line 2.**
- ○ **It will not crash, and nothing will print.**

Submit

Debugging Functions with Print Statements

We have one last topic before you do the section exercises. It's really just a strong suggestion. As you write increasingly complex functions, they will become increasingly hard to debug. One super helpful technique is to add print statements inside your function that print the values of the relevant local variables. Then, run the test case that fails, and look very closely at the output, line by line. Hopefully, one line will surprise you, and it will probably give you strong insights as to where the bug is. This is extremely effective.

Here are some helpful hints:

1. When you use print statements to debug, it can be hard to keep track of which variable is on which line of output. This is fixed by printing not just the variable, but also its name. That is, instead of `print(x)`, try using `print('x:', x)`.
2. Be sure to remove your debugging print statements when you are ready to submit the code to the autograder.

With that, good luck, and have fun with the exercises!

Continue